

02\_Sum\_of\_First\_N\_Numbers.cpp U 03\_Power\_of\_A\_to\_B

Source Code > 02\_Sum\_of\_First\_N\_Numbers.cpp > main()

```
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     int n;
7     cin >> n;
8
9     int sum = 0;
10
11     for(int i = 1; i <= n; i++)
12     {
13         sum += i;
14     }
15
16     cout << sum;
17
18     return 0;
19 }
```

TERMINAL

Code - Source\_Code

```
PS C:\Users\omsin\OneDrive\Desktop\3rd-Year-Week-4-Module\Source_Code> cd "c:\Users\omsin\OneDrive\Desktop\3rd-Year-Week-4-Module\Source_Code" ; if ($?) { g++ 02_Sum_of_First_N_Numbers.cpp -o 02_Sum_of_First_N_Numbers } ; if ($?) {
Sum_of_First_N_Numbers }
5
15
PS C:\Users\omsin\OneDrive\Desktop\3rd-Year-Week-4-Module\Source_Code>
```

Source\_Code > 03\_Power\_of\_A\_to\_B.cpp > main()

```
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     int a, b;
7     cin >> a >> b;
8
9     int ans = 1;
10
11     for(int i = 1; i <= b; i++)
12     {
13         ans = ans * a;
14     }
15
16     cout << ans;
17
18     return 0;
19 }
```

TERMINAL

```
PS C:\Users\omsin\OneDrive\Desktop\3rd-Year-Week-4-Module> cd "c:\Users\omsin\OneDrive\Desktop\Source_Code\" ; if ($?) { g++ 03_Power_of_A_to_B.cpp -o 03_Power_of_A_to_B } ; if ($?) { .\03_Power_of_A_to_B }
3
5
243
PS C:\Users\omsin\OneDrive\Desktop\3rd-Year-Week-4-Module\Source_Code>
```

```
Source_Code > 01_Print_1_to_N.cpp > main()
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int n;
7      cin >> n;
8
9      for(int i = 1; i <= n; i++)
10     {
11         cout << i << " ";
12     }
13
14     return 0;
15 }
```

```
PS C:\Users\omsin\OneDrive\Desktop\3rd-Year-Week-4-Module\Source_Code>
cd "c:\Users\omsin\OneDrive\Desktop\3rd-Year-Week-4-Module\Source_Code\" ; if ($?) { g++ 01_Print_1_to_N.cpp -o 01_Print_1_to_N } ; if ($?) { .\01_Print_1_to_N }
1 2 3 4 5
PS C:\Users\omsin\OneDrive\Desktop\3rd-Year-Week-4-Module\Source_Code>
```

## ★ Time Complexity

Time Complexity is the total time taken by an algorithm to run as a function of the input size ( $n$ ). It helps us to analyse the efficiency of an algorithm.

### Big O Notation ( $O$ )

⇒ Big O notation is used to represent the upper bound of time complexity.

It tells us the worst case time complexity.

It ignores constant factors and lower order terms.

Example →  $T(n) = 3n^2 + 5n + 7 \Rightarrow O(n^2)$   
(highest power)

### Cases in Time Complexity :-

- 1) Worst Case → Maximum time taken by an algorithm  
It gives the upper bound.
- 2) Average Case → Average time taken over all possible inputs of size  $n$ .
- 3) Best Case → Minimum time taken by an algorithm  
It gives the lower bound.

### Common Time Complexities :-

- |                 |                     |
|-----------------|---------------------|
| → $O(1)$        | → Constant Time     |
| → $O(\log n)$   | → Logarithmic Time  |
| → $O(n)$        | → Linear Time       |
| → $O(n \log n)$ | → Linearithmic Time |
| → $O(n^2)$      | → Quadratic Time    |
| → $O(2^n)$      | → Exponential Time  |

### order of growth (increasing)

$O(1) < O(\log n) < O(n) <$   
 $O(n \log n) < O(n^2) <$   
 $O(2^n) < O(n!)$

(faster to slower)

Note → We always try to design algorithms with lower time complexity.





## Space Complexity

Space Complexity is the total memory used by an algorithm to ~~run~~ execute as a function of input size ( $n$ ).

It includes space used by variables, data structures, ~~and~~ function call stack, etc.

### Types of Space :-

- 1) Auxiliary Space  $\rightarrow$  Extra space used by the algorithm.
- 2) Total Space  $\rightarrow$  Input space + Auxiliary Space.

### Common Space Complexities :-

- 1)  $O(1)$   $\rightarrow$  Constant Space  
Eg. `int a = 5;`
- 2)  $O(\log n)$   $\rightarrow$  Logarithmic Space  
Eg. In recursion (binary search)
- 3)  $O(n)$   $\rightarrow$  Linear Space  
Eg. Array of size  $n$
- 4)  $O(n \log n)$   $\rightarrow$  Linearithmic Space  
Eg. Merge sort (~~extra~~ array)
- 5)  $O(n^2)$   $\rightarrow$  Quadratic Space  
Eg. 2D array of size  $n \times n$
- 6)  $O(2^n)$   $\rightarrow$  Exponential Space  
Eg. recursive solutions (fibonacci naive)

### Examples :-

- |                                |                      |
|--------------------------------|----------------------|
| 1) <code>int a = 10;</code>    | $\rightarrow O(1)$   |
| 2) <code>int arr[n];</code>    | $\rightarrow O(n)$   |
| 3) <code>int mat[n][n];</code> | $\rightarrow O(n^2)$ |

Note  $\rightarrow$  Always try to ~~opt~~ optimize space.  
sometimes time ke saath space bhi ~~badh~~ jata hai.

★

Question 2 :-

Write a program to find sum of first  $N$  natural numbers.

Algorithm :-

1. Start
2. Read  $N$
3.  $sum = 0$ ,  $i = 1$
4. Repeat steps 5 to 6 while  $i \leq N$
5.  $sum = sum + i$
6.  $i = i + 1$
7. Print  $sum$
8. Stop

Dry Run :-

| $N$ | $i$ (initial) | $sum$ (initial) | $sum$ (update) | $i$ (update) | Condition                |
|-----|---------------|-----------------|----------------|--------------|--------------------------|
| 5   | 1             | 0               | 1              | 2            | $1 \leq 5 \rightarrow T$ |
|     | 2             | 1               | 3              | 3            | $2 \leq 5 \rightarrow T$ |
|     | 3             | 3               | 6              | 4            | $3 \leq 5 \rightarrow T$ |
|     | 4             | 6               | 10             | 5            | $4 \leq 5 \rightarrow T$ |
|     | 5             | 10              | 15             | 6            | $5 \leq 5 \rightarrow T$ |
|     | 6             | 15              | 15             | 6            | $6 \leq 5 \rightarrow F$ |

Stop

Code :-

```
#include <iostream>
using namespace std;
int main() {
    int n, i = 1, sum = 0;
    cin >> n;
    while (i <= n) {
        sum = sum + i;
        i = i + 1;
    }
    cout << sum;
    return 0;
}
```

Time Complexity :-  $O(n)$ Space Complexity :-  $O(1)$ Where  $n$  = input numberSample Input / Output :-

Input : 5

Output : 15





### Question 3 :-

Write a program to find power of  $A^B$ .

#### Algorithm :-

1. Start
2. Read A and B
3. Initialize result = 1
4.  $i = 1$
5. Repeat steps 6 to 7 while  $i \leq B$
6.  $result = result * A$
7.  $i = i + 1$
8. Print result
9. Stop

#### Dry Run :-

| A | B | i (initial) | result(initial) | i (update) | result(update) | Condition                |
|---|---|-------------|-----------------|------------|----------------|--------------------------|
| 2 | 5 | 1           | 1               | 2          | 2              | $1 \leq 5 \rightarrow T$ |
|   |   | 2           | 2               | 3          | 4              | $2 \leq 5 \rightarrow T$ |
|   |   | 3           | 4               | 4          | 8              | $3 \leq 5 \rightarrow T$ |
|   |   | 4           | 8               | 5          | 16             | $4 \leq 5 \rightarrow T$ |
|   |   | 5           | 16              | 6          | 32             | $5 \leq 5 \rightarrow T$ |
|   |   | 6           | 32              | -          | -              | $6 \leq 5 \rightarrow F$ |

Stop.

#### Code :-

```
#include <iostream>
using namespace std;
int main() {
    int A, B;
    cout << "Enter A and B:";
    cin >> A >> B;
    int result = 1;
    for (int i = 1; i <= B; i++) {
        result = result * A;
    }
    cout << "Power is: " << result;
    return 0;
}
```

#### Time Complexity :-

$O(B)$

(loop runs B times)

#### Space Complexity :-

$O(1)$

Where A = base

B = exponent

#### Sample Input / Output :-

Input : A = 2 , B = 5

Output : 32



### Question 1 :-

Write a program to print numbers from 1 to N.

### Algorithm :-

1. Start
2. Read N
3.  $i = 1$
4. Repeat steps 5 to 6 while  $i \leq N$
5. Print i
6.  $i = i + 1$
7. Stop

### Dry Run :-

| N | i (initial) | Printed   | i (update) | Condition                |
|---|-------------|-----------|------------|--------------------------|
| 5 | 1           | 1         | 2          | $1 \leq 5 \rightarrow T$ |
|   | 2           | 1 2       | 3          | $2 \leq 5 \rightarrow T$ |
|   | 3           | 1 2 3     | 4          | $3 \leq 5 \rightarrow T$ |
|   | 4           | 1 2 3 4   | 5          | $4 \leq 5 \rightarrow T$ |
|   | 5           | 1 2 3 4 5 | 6          | $5 \leq 5 \rightarrow T$ |
|   |             |           | 6          | $6 \leq 5 \rightarrow F$ |

Stop

### Code :-

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    for(int i=1; i<=n; i++) {
        cout << i << " ";
    }
    return 0;
}
```

### Time Complexity :-

$O(n)$

### Space Complexity :-

$O(1)$

Where  $n = \text{input number}$

### Sample Input / Output :-

Input : 5

Output : 1 2 3 4 5